CS6160 Complexity

Topic: Space Complexity. QBF. Savitch's Theorem. Log Space. NL = coNL. Date: Sep 16, 2026
Lecturer: Wei-Kai Lin Scriber: Nathaneal Wattier

---

1 Quantified Boolean Formulas

We begin with a generalization of the SAT problem: Quantified Boolean Formulas. Just as SAT captures the essence of NP, so too will QBF capture PSPACE. A QBF consists of a Boolean formula φ preceded by an arbitrary number of $\exists$ and $\forall$ quantifiers. It is not too hard to see that, by inserting dummy quantified variables that do not appear in φ, any QBF can be put in alternating form, i.e. $\exists x_1 \forall x_2 \exists x_3 \ldots$.

Definition 1 (QBF)**.** A Quantified Boolean Formula is an expression

$$Q_1 x_1 \, Q_2 x_2 \cdots Q_n x_n \; \varphi(x_1, \ldots, x_n), \qquad Q_i \in \{\exists, \forall\},$$

where φ is a Boolean formula. Since every variable is quantified, the expression is either true or false. TQBF is the language of true QBFs.

In fact, TQBF is PSPACE-complete. A common way to look at QBF is through the lens of game playing, using the alternating form. Two players take turns: P_1 picks x_1, P_2 picks x_2, and so on, and P_1 wins if the final assignment satisfies φ. The QBF is true if and only if P_1 has a winning strategy, i.e., there exists a first move for P_1 such that for every second move by P_2 there exists a third move for P_1, and so forth, ending with φ satisfied.

2 Savitch's Theorem

Theorem 2 (Savitch)**.** *For any space-constructible $S(n) \geq \log n$, we have* $\mathsf{NSPACE}(S(n)) \subseteq \mathsf{SPACE}(S(n)^2)$.

Proof Sketch. Let M be a nondeterministic TM using $S(n)$ space. A configuration of M on input x (state, head positions, work tape contents) takes $O(S(n))$ bits, so there are at most $2^{cS(n)}$ configurations for some constant c. This is where $S(n) \geq \log n$ is needed, since just the input head position takes $\log n$ bits. M accepts x if and only if an accepting configuration is reachable from the start configuration in at most $2^{cS(n)}$ steps.

Let $\textsc{Reach}(c_1, c_2, i)$ be true if and only if c_2 is reachable from c_1 in at most 2^i steps. For $i = 0$ this is checked directly from the transition function, and for $i > 0$

$$\textsc{Reach}(c_1, c_2, i) = \exists c_m : \textsc{Reach}(c_1, c_m, i-1) \wedge \textsc{Reach}(c_m, c_2, i-1).$$

A deterministic machine evaluates this by trying every midpoint c_m and making the two recursive calls one after the other, reusing the same space. The recursion has depth $cS(n)$ and each level stores $O(S(n))$ bits (c_1, c_2, c_m, i), for $O(S(n)^2)$ space in total. □

Corollary 3. $\mathsf{PSPACE} = \mathsf{NPSPACE}$*, since*

$$\mathsf{PSPACE} \subseteq \mathsf{NPSPACE} = \bigcup_{c>0} \mathsf{NSPACE}(n^c) \subseteq \bigcup_{c>0} \mathsf{SPACE}(n^{2c}) = \mathsf{PSPACE}.$$

3 Log Space

Definition 4 (L and NL)**.** We define $\mathsf{L} = \mathsf{SPACE}(\log n)$ and $\mathsf{NL} = \mathsf{NSPACE}(\log n)$. Here the input is on a read-only tape, and only the work tape counts toward the space bound.

We care about L because in many applications space and memory are the predominant constraints. Oftentimes there is a natural tradeoff between the space and time an algorithm uses. For instance, as we will see later in the course, SAT cannot be solved in linear time and logarithmic space simultaneously.

By Savitch's theorem, $\mathsf{L} \subseteq \mathsf{NL} \subseteq \mathsf{SPACE}(\log^2 n)$. By the space hierarchy theorem $\mathsf{L} \subsetneq \mathsf{SPACE}(\log^2 n)$, so at least one of these inclusions is strict; we will see more about this later in the course.

4 Complement Classes

Definition 5 (Complement Class)**.** Let $S \subseteq \{f : \{0,1\}^* \to \{0,1\}\}$. We denote by $\mathsf{co}S$ the class $\{\bar{f} : f \in S\}$.

For example:

- P is closed under complement, so $\mathsf{coP} = \mathsf{P}$.
- It is a major open question whether $\mathsf{NP} = \mathsf{coNP}$.
- The problem of deciding whether a Boolean formula is true under every assignment is known as TAUTOLOGY. It is the analogue of SAT for coNP, and is coNP-complete.
- A polynomial-time reduction between TAUTOLOGY and SAT (in either direction) would prove $\mathsf{NP} = \mathsf{coNP}$.

Definition 6 (coNP, alternative definition)**.** $F \in \mathsf{coNP}$ if and only if there exist a polynomial-time TM V and a polynomial p such that, for $n = |x|$,

$$F(x) = 1 \iff \forall w \in \{0,1\}^{p(n)} \ V(x,w) = 1.$$

In NP we interpret w as a witness. Here, instead of requiring that some witness exists, we require that $V(x,w) = 1$ for every w.

Theorem 7. $\mathsf{coNL} = \mathsf{NL}$.

The proof is given at the end of the next section.

5 PATH

Definition 8 (PATH)**.** For a directed graph G and vertices s, t,

$$\text{PATH}(G,s,t) = \begin{cases} 1 & \text{if there exists a path from } s \text{ to } t \text{ in } G, \\ 0 & \text{otherwise.} \end{cases}$$

PATH is in P (e.g. by BFS) and PATH $\in \mathsf{NL}$ since a nondeterministic machine guesses the path one vertex at a time, storing the current vertex and a step counter up to the number of vertices, which takes $O(\log n)$ bits.

Definition 9 (Log-space reduction, NL-completeness)**.** A log-space reduction from F to F' is a function R computed by a deterministic log-space TM such that $F(x) = F'(R(x))$. The output of R is written on a write-only output tape that does not count toward its space. F' is NL-complete if $F' \in \mathsf{NL}$ and every $F \in \mathsf{NL}$ has a log-space reduction to F'.

Theorem 10. PATH *is* NL*-complete.*

Proof Sketch. We showed PATH $\in \mathsf{NL}$ above. Let $F \in \mathsf{NL}$ be decided by a nondeterministic TM M in space $O(\log n)$. WLOG, M has a unique accepting configuration. On input x, R outputs $(G_{M,x}, c_{\text{start}}, c_{\text{acc}})$, where $G_{M,x}$ is the configuration graph of M on x: its vertices are configurations, with an edge $c \to c'$ if M can move from c to c' in one step. Each configuration takes $O(\log n)$ bits, so R can loop over all pairs (c, c') and check the transition function (reading x as needed) in $O(\log n)$ space. The output is polynomially long, but it goes on the output tape and is not counted. Then $F(x) = \text{PATH}(R(x))$. □

Proof of Theorem 7. Every $F \in \mathsf{NL}$ reduces to PATH, so every $F \in \mathsf{coNL}$ reduces to $\overline{\text{PATH}}$ by the same reductions, and $\overline{\text{PATH}}$ is coNL-complete. Since NL is closed under log-space reductions, it suffices to show $\overline{\text{PATH}} \in \mathsf{NL}$, or else to give a polynomial-size certificate that t is not reachable from s which a log-space verifier can check in one pass. This seems quite difficult at first, but in fact there is a short certificate for a path not existing.

Now we will construct such a certificate of polynomial length in n, which can be verified in logarithmic space. Let G have n vertices and let S_i be the set of vertices reachable from s in at most i steps. Then $S_0 = \{s\}$, and t is unreachable if and only if $t \notin S_{n-1}$. We build certificates for three kinds of statements:

1. $v \in S_i$**:** a path $s = v_0, v_1, \ldots, v_k = v$ with $k \le i$. This is the same as for PATH.
2. $v \notin S_i$ **given** $|S_{i-1}|$**:** every vertex $u \in S_{i-1}$ listed in increasing order, each with a certificate that $u \in S_{i-1}$. The verifier checks that the list is increasing, that its length is $|S_{i-1}|$, and that every listed u satisfies $u \neq v$ and $(u, v) \notin E$. Since $v \in S_i$ if and only if $v \in S_{i-1}$ or v has an in-neighbor in S_{i-1}, this proves $v \notin S_i$.
3. $|S_i| = c$ **given** $|S_{i-1}|$**:** for each vertex $v = 1, \ldots, n$ in order, a certificate that $v \in S_i$ or that $v \notin S_i$. The verifier counts the vertices certified to be in S_i and checks that the count is c.

The full certificate for $\overline{\text{PATH}}(G, s, t)$ consists of the third type of certificate for $|S_1|, |S_2|, \ldots, |S_{n-1}|$, each checked using the one before it. Its length is polynomial in n. The verifier reads it in one pass and only stores a constant number of vertices and counters, each $O(\log n)$ bits, so it runs in logarithmic space. □